

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE
SOFTWARE

Informe de Investigación y Práctica Experimental

Unidad III — Gestión de Estado, Acceso a Datos y Arquitecturas Web
Escalables Objetos integrados y gestión de estado, acceso a datos
desde aplicaciones web, patrones de arquitectura y diseño de
arquitecturas web escalables

Asignatura:

Aplicaciones Web — 5to Nivel «A»

Docente:

Ing. Gleiston Cicerón Guerrero Ulloa

Integrantes:

Johan Stalin Carvajal Loor
Michael Xavier Fajardo Montes Jaime
Josué Mariscal Cabrera

Período Académico 2026–2027 PPA
Quevedo — Los Ríos — Ecuador

1 Introducción

La programación del lado del servidor no se limita a recibir una petición y devolver una respuesta: una aplicación web real necesita recordar quién es el usuario a lo largo de varias peticiones, comunicarse de forma eficiente y segura con una base de datos, y estar organizada de tal manera que pueda crecer sin volverse imposible de mantener. Estos tres problemas —la gestión de estado, el acceso a datos y la organización arquitectónica— son el hilo conductor de esta unidad.

El informe se organiza en cuatro temas. El primero explica los objetos integrados que las distintas tecnologías de servidor ofrecen para representar una petición, una respuesta, y el estado de un usuario o de la aplicación completa. El segundo tema revisa las tecnologías disponibles para que el backend se comunique con una base de datos, desde el uso directo de SQL hasta el mapeo objeto-relacional. El tercer tema aborda los principios y patrones de arquitectura de software aplicados a aplicaciones web, incluyendo la arquitectura por capas y alternativas como los microservicios y las arquitecturas orientadas a eventos. Finalmente, el cuarto tema profundiza en el diseño de arquitecturas pensadas para escalar, cubriendo patrones de caché y la documentación de decisiones arquitectónicas. Cada tema se apoya en documentación oficial de las tecnologías involucradas, citada en formato IEEE a lo largo del texto.

2 Tema 1: Objetos Integrados y Gestión de Estado en Aplicaciones Web

Toda aplicación web necesita una forma de recibir información del usuario, devolverle una respuesta, y recordar quién es a lo largo de varias peticiones, aunque el protocolo HTTP en sí no recuerde nada de una petición a otra. Para resolver esto, las distintas plataformas de programación del lado del servidor ofrecen un conjunto de objetos integrados que representan cada una de esas necesidades. Aunque el nombre y la forma exacta cambian según la tecnología, el concepto detrás es el mismo en PHP, Java y ASP.NET.

2.1 El objeto Request

Cuando un navegador hace una petición HTTP, en realidad está enviando texto plano con un formato específico: una línea con el método y la ruta, seguida de cabeceras, y opcionalmente un cuerpo con datos. Procesar ese texto a mano en cada aplicación sería repetitivo y propenso a errores, por lo que las plataformas de servidor lo empaquetan automáticamente en un objeto estructurado antes de que el código del desarrollador siquiera se ejecute. En Java, ese objeto es `HttpServletRequest`: el contenedor de servlets lo construye a partir de la petición cruda y lo entrega como parámetro a los métodos que la atienden, como `doGet` o `doPost`, exponiendo métodos como `getParameter()` para leer datos de un formulario, `getHeader()` para leer una cabecera específica, o `getMethod()` para saber si la petición fue GET o POST [1]. En PHP no existe un único objeto Request; en su lugar, esa misma información se reparte entre varias variables superglobales que PHP llena automáticamente al iniciar cada script: `$_GET` recoge los datos que vienen en la URL, `$_POST` los que vienen en el cuerpo de un formulario, y `$_SERVER` agrupa detalles técnicos de la petición y del entorno del servidor [2]. En ASP.NET Core, toda esa información vuelve a unificarse en un solo lugar, accesible desde cualquier controlador a través de la propiedad `HttpContext.Request`. En los tres casos, el propósito es el mismo: esconder la complejidad del protocolo HTTP crudo detrás de una interfaz simple que el desarrollador pueda consultar directamente.

2.2 El objeto Response

El objeto Response cumple el papel opuesto: representa lo que el servidor va a devolver al navegador una vez que terminó de procesar la petición. Esto incluye el código de estado HTTP (como 200 para éxito o 404 para no encontrado), las cabeceras de la respuesta, y el contenido que se va a mostrar. En Java, `HttpServletResponse` permite definir el tipo de contenido, escribir la salida a través de un `PrintWriter`, y fijar cabeceras o el código de estado; un detalle técnico importante es que, una vez que el servidor ya empezó a enviar la respuesta al navegador —lo que se conoce como que la respuesta ya fue "confirmada" o `committed`—, ya no es posible modificar las cabeceras ni el código de estado, porque esa parte ya salió [1]. Esto obliga a que toda la lógica de decisión (por ejemplo, decidir si algo salió bien o mal) ocurra antes de empezar a escribir el cuerpo de la respuesta. En PHP ocurre algo parecido: todo lo que el script imprime se convierte en el cuerpo de la respuesta, y la función `header()` debe llamarse antes de que se haya impreso cualquier contenido, o PHP genera un error, reflejando la misma restricción de fondo. En ASP.NET Core, la propiedad `HttpContext.Response` ofrece el mismo control que su equivalente en Java.

2.3 El objeto Session

HTTP es un protocolo sin memoria: cada petición es independiente de la anterior, y por sí solo el servidor no tiene forma de saber si dos peticiones distintas vienen del mismo usuario. El objeto Session existe para resolver justamente ese problema, permitiendo que el servidor guarde datos asociados a un usuario específico mientras dura su visita al sitio [3]. El mecanismo es prácticamente idéntico en todas las tecnologías: el servidor genera un identificador único de sesión y se lo entrega al navegador mediante una cookie; en cada petición siguiente, el navegador reenvía esa cookie, y el servidor la usa para recuperar los datos guardados de ese usuario en particular [3]. En Java se accede mediante HttpSession, obtenido llamando a `request.getSession()` [1]; en PHP se usa la superglobal `$_SESSION`, después de iniciar la sesión con `session_start()` [2]; y en ASP.NET Core se configura como un servicio respaldado por una caché, con un tiempo de expiración por defecto de 20 minutos de inactividad [3]. Un principio de diseño importante, válido en las tres tecnologías, es que los datos de sesión deben tratarse como información temporal y no crítica: la aplicación debería poder seguir funcionando razonablemente si esos datos se pierden, y la información realmente importante (como los datos de un pedido o de una cuenta) debe vivir siempre en la base de datos, no únicamente en la sesión [3]. Además, como la sesión viaja mediante una cookie que suele identificar a una persona real, su manejo está sujeto a consideraciones de privacidad como las del RGPD europeo [3].

2.4 Los objetos Application y Page

Mientras que Session guarda datos de un usuario particular, el objeto Application (o su equivalente) guarda datos compartidos por todos los usuarios de la aplicación al mismo tiempo, sin importar quién esté conectado en ese momento. En Java, el equivalente es el ServletContext: un único objeto por aplicación, accesible desde todos los servlets, típicamente usado para configuración global o recursos de solo lectura que no cambian según el usuario, como parámetros de inicialización [1]. Es importante no confundir este ámbito con el de la sesión: como Application es compartido por todas las peticiones concurrentes de todos los usuarios, guardar ahí datos que cambian por usuario puede generar condiciones de carrera si dos peticiones intentan modificar el mismo dato al mismo tiempo. En ASP.NET, este concepto existió históricamente como estado de aplicación, aunque la documentación actual de Microsoft recomienda usar en su lugar una caché o una base de datos para la mayoría de los casos, reservando el estado de aplicación puro para escenarios muy puntuales de solo lectura.

Por último, el alcance de Page es el más restringido de todos: se refiere a datos que solo existen durante el procesamiento de una única página o petición, y que se descartan apenas termina de generarse la respuesta. Este concepto es especialmente visible en JSP, donde cada página tiene acceso a un objeto llamado `pageContext` que permite consultar los cuatro ámbitos posibles para un dato —page, request, session y application—, ordenados de menor a mayor alcance. El ámbito page es el más corto de todos: ni siquiera sobrevive si la petición involucra a otro componente además de la página actual, lo que lo hace útil solamente para datos auxiliares de un único renderizado

3 Tema 2: Acceso a Datos desde Aplicaciones Web

Casi ninguna aplicación web tiene sentido sin una base de datos detrás: el backend necesita guardar, consultar y modificar información de forma persistente, mucho después de que la petición HTTP que la originó ya terminó. Pero el lenguaje de programación del backend y el motor de base de datos son dos mundos distintos, que hablan protocolos distintos, y conectarlos de forma segura y eficiente requiere tecnologías específicas. Este tema revisa esas tecnologías, desde el acceso más directo con SQL hasta las capas de abstracción que evitan escribirlo a mano.

3.1 Tecnologías de acceso a datos en aplicaciones web

Cada plataforma de programación del lado del servidor ofrece su propia forma estándar de conectarse a una base de datos relacional. En Java, esa forma es JDBC (Java Database Connectivity), una API que permite que el código Java se conecte a prácticamente cualquier motor de base de datos a través de un driver específico para ese motor, ejecute sentencias SQL, y reciba los resultados como objetos que el propio programa puede recorrer, como un `ResultSet` [4]. Lo importante de JDBC es que el código de la aplicación no necesita saber los detalles internos de comunicación con la base de datos: solo necesita el driver correcto y una cadena de conexión, y toda la complejidad de hablar el protocolo nativo del motor queda oculta detrás de esa API común. En PHP existe un equivalente llamado PDO (PHP Data Objects), una extensión que ofrece una interfaz consistente para trabajar con distintos motores de base de datos —MySQL, PostgreSQL, SQLite, entre otros— usando prácticamente el mismo código, de modo que cambiar de motor de base de datos implica modificar poco más que la cadena de conexión [5]. En ambos casos, la idea de fondo es la misma: aislar al desarrollador de los detalles particulares del motor de base de datos que se esté usando en ese momento.

3.2 Consultas SQL desde aplicaciones web

Enviar una consulta SQL desde una aplicación web no es tan simple como pegar el texto del SQL directamente, sobre todo cuando esa consulta depende de datos que ingresó el usuario, como un nombre de búsqueda o un identificador. Construir la consulta concatenando ese dato directamente dentro del texto SQL abre la puerta a la inyección SQL, uno de los riesgos de seguridad más conocidos en aplicaciones web. La forma correcta y estándar de evitarlo es mediante sentencias preparadas: primero se define la estructura de la consulta con espacios reservados para los valores variables, y luego esos valores se envían por separado al motor de base de datos, que nunca los interpreta como parte del código SQL, sino solamente como datos. En JDBC, esto se logra con un objeto `PreparedStatement`, creado a partir de `connection.prepareStatement()`, donde los valores se asignan uno por uno con métodos como `setString()` o `setInt()` antes de ejecutar la consulta [4]. En PDO, el mismo patrón se logra con `prepare()`, `bindValue()` y `execute()` [5]. Además de la seguridad, las sentencias preparadas también ofrecen una ventaja de rendimiento cuando la misma consulta se ejecuta muchas veces con distintos valores, porque el motor de base de datos puede reutilizar el plan de ejecución ya calculado en lugar de recompilarlo cada vez.

3.3 Procedimientos almacenados y funciones de BD

Un procedimiento almacenado o una función de base de datos es un bloque de lógica que se guarda directamente dentro del motor de base de datos, en lugar de vivir en el código de la aplicación, y que se invoca por su nombre en vez de enviar el texto completo de la consulta cada vez. En PostgreSQL, la distinción entre ambos es concreta: una función devuelve un valor y puede usarse dentro de otras expresiones, como en una cláusula `SELECT` o `WHERE`, mientras que un procedimiento no devuelve ningún valor y se ejecuta con la

instrucción CALL; además, solo los procedimientos pueden controlar transacciones completas dentro de su propio cuerpo, haciendo COMMIT o ROLLBACK, algo que una función no puede hacer [6]. La principal ventaja de mover lógica al motor de base de datos es que esa lógica queda centralizada en un solo lugar: si varias aplicaciones distintas acceden a la misma base de datos, todas comparten exactamente las mismas reglas, sin duplicar validaciones en cada aplicación por separado, y se reduce la cantidad de datos que viajan por la red en cada operación. La desventaja es que esa lógica queda atada a un motor de base de datos específico, es más difícil de probar y versionar con las mismas herramientas que se usan para el resto del código de la aplicación, y dificulta escalar o migrar el backend de forma independiente de la base de datos.

3.4 ORM y mapeo objeto-relacional

El mapeo objeto-relacional (ORM) nace de un problema de fondo conocido como impedance mismatch: el código orientado a objetos trabaja con clases, herencia y referencias entre objetos, mientras que una base de datos relacional trabaja con tablas, filas y claves foráneas, y estos dos modelos no encajan perfectamente entre sí. Un ORM es la herramienta que traduce automáticamente entre ambos mundos, permitiendo que el desarrollador trabaje con objetos normales de su lenguaje de programación mientras el ORM genera por detrás el SQL necesario para guardar o recuperar esos objetos de la base de datos. En el ecosistema Java, el estándar para esto es JPA (Jakarta Persistence API): una especificación, no una implementación concreta, que define anotaciones como @Entity para marcar que una clase corresponde a una tabla, y una API llamada EntityManager para guardar, consultar, actualizar o eliminar esos objetos sin escribir SQL directamente [7]. Como toda especificación, JPA necesita un proveedor que la implemente realmente; Hibernate es, en la práctica, el proveedor más usado del ecosistema Java para cumplir esa especificación. La ventaja de un ORM es velocidad de desarrollo para las operaciones CRUD típicas, ya que evita escribir SQL repetitivo a mano; la desventaja aparece en consultas más complejas o de reportes, donde el SQL generado automáticamente puede ser menos eficiente que uno escrito directamente, una consideración de rendimiento que conecta directamente con los patrones de caché que se revisan en el Tema 4.

4 Tema 3: Patrones de Arquitectura para Aplicaciones Web

La arquitectura de software determina qué tan fácil (o difícil) será mantener y hacer crecer una aplicación web con el tiempo, mucho más allá de si el código funciona correctamente en el momento en que se escribe. Este tema revisa los principios generales detrás de esa disciplina, y las principales formas de organizar un sistema, desde la arquitectura por capas —la más común en aplicaciones web tradicionales— hasta alternativas como los microservicios o las arquitecturas orientadas a eventos.

4.1 Principios de arquitectura de software

La arquitectura de software se define como el conjunto de decisiones estructurales fundamentales sobre cómo se organiza un sistema: qué elementos lo componen, cómo se relacionan entre sí, y qué principios guían su diseño y evolución [8]. Estas decisiones determinan atributos de calidad como el rendimiento, la seguridad, la mantenibilidad y la capacidad de escalar, y son mucho más difíciles de revertir que un simple error de código: un error arquitectónico temprano, como acoplar excesivamente la lógica de negocio con el acceso a datos, puede volverse costoso de corregir a medida que el sistema crece [8]. Por eso, la arquitectura no es un diagrama que se dibuja una vez al inicio del proyecto, sino un conjunto de decisiones que condicionan tanto la velocidad de desarrollo del equipo como la facilidad con la que se pueden incorporar nuevas funcionalidades sin romper las existentes.

4.2 Arquitectura por capas

La arquitectura por capas organiza el sistema en niveles horizontales, donde cada capa solo se comunica con la inmediata inferior, típicamente separando presentación, lógica de negocio y acceso a datos [9]. Garlan y Shaw señalan que este estilo tiene ventajas concretas: soporta el diseño basado en niveles crecientes de abstracción, lo que permite dividir un problema complejo en pasos incrementales; facilita mejoras porque cada capa solo interactúa con las capas inmediatamente adyacentes, de modo que un cambio afecta como máximo a dos niveles; y favorece la reutilización, ya que distintas implementaciones de una misma capa pueden intercambiarse siempre que respeten la misma interfaz hacia las capas vecinas [9]. La desventaja principal es que no todos los sistemas se estructuran naturalmente en capas limpias, y a veces las consideraciones de rendimiento obligan a acoplar más de lo ideal una función de alto nivel con su implementación de bajo nivel [9]. En una aplicación web típica, esto se traduce en Controladores que reciben las peticiones HTTP, Servicios que contienen las reglas de negocio, y Repositorios que encapsulan el acceso a la base de datos.

4.3 Arquitecturas en pizarra, dirigida por eventos y peer-to-peer

El estilo de pizarra (blackboard) organiza el sistema alrededor de un repositorio de datos central y compartido, donde varios componentes independientes —llamados fuentes de conocimiento— se activan automáticamente cuando el dato que les interesa cambia, de forma similar a como funciona una base de datos activa que invoca componentes externos según qué información se modificó [9]. Este estilo es útil cuando ningún algoritmo único conoce la solución completa de un problema, y distintos componentes especializados deben ir aportando piezas de la solución de forma oportunista.

La arquitectura orientada a eventos, en cambio, organiza el sistema alrededor de productores que generan eventos cuando ocurre un cambio de estado relevante, y consumidores que reaccionan a esos eventos de forma asíncrona, comunicados a través de un intermediario (event bus o broker) que los desacopla en el tiempo y en el espacio. A diferencia del modelo REST síncrono, donde el cliente espera una respuesta inmediata, este estilo permite que productor y consumidor operen de forma independiente, mejorando la resiliencia ante picos de carga a costa de mayor complejidad para razonar sobre el orden de los eventos.

Las arquitecturas peer-to-peer (P2P), por su parte, eliminan la distinción entre cliente y servidor: cada nodo de la red puede actuar simultáneamente como consumidor y proveedor de un recurso, comunicándose directamente con otros nodos sin depender de un servidor central único. Esto elimina el punto único de fallo que representa un servidor central, pero introduce el desafío de coordinar nodos que pueden desconectarse en cualquier momento, sin que ningún nodo tenga una visión completa y actualizada del sistema.

4.4 Arquitectura Orientada a Servicios (SOA) y Microservicios

La Arquitectura Orientada a Servicios (SOA) es un marco conceptual que organiza las capacidades de un sistema como servicios independientes, entendidos como mecanismos para proporcionar acceso a una o más capacidades, donde ese acceso se da a través de una interfaz definida y se ejerce de forma consistente con las restricciones y políticas especificadas por la descripción del servicio [10]. El modelo de referencia de OASIS para SOA es deliberadamente abstracto: no está atado a ninguna tecnología o estándar concreto, sino que busca dar un vocabulario común para razonar sobre sistemas orientados a servicios independientemente de cómo se implementen [10].

Los microservicios son, en cierto sentido, una evolución concreta y más granular de esa misma idea: dividen el sistema en servicios pequeños e independientes, cada uno con su propia base de datos y ciclo de despliegue propio, que se comunican entre sí por red [10].

Esto permite escalar cada servicio por separado y que distintos equipos trabajen sin bloquearse mutuamente, pero introduce complejidad operativa considerable: hay que gestionar comunicación entre servicios, consistencia de datos distribuidos, y tolerancia a fallos de red que en un sistema monolítico simplemente no existen [11]. Por esta razón, tanto SOA como los microservicios se justifican principalmente cuando el tamaño del equipo o del sistema ya no puede coordinarse eficientemente dentro de una sola base de código compartida; para proyectos pequeños o de alcance acotado, la arquitectura por capas dentro de un único despliegue suele ser la opción más razonable.

5 Tema 4: Diseño de Arquitecturas Web Escalables

Una aplicación web que funciona bien con diez usuarios de prueba no necesariamente funciona igual de bien con diez mil usuarios reales al mismo tiempo. Diseñar pensando en la escalabilidad significa anticipar ese crecimiento desde el diseño, no como un parche posterior. Este tema cierra el informe revisando cómo escalar una aplicación web, qué papel juega la caché en ese proceso, y cómo se documentan las decisiones arquitectónicas para que sigan siendo comprensibles con el tiempo.

5.1 Escalabilidad en aplicaciones web

Escalar un sistema significa aumentar su capacidad para atender más carga de trabajo, y existen dos estrategias principales para lograrlo. La escalabilidad vertical consiste en aumentar los recursos de un único servidor —más memoria, más procesador—, una solución simple pero con un límite físico y de costo. La escalabilidad horizontal, en cambio, consiste en agregar más instancias del mismo servidor y repartir la carga entre ellas mediante un balanceador, una estrategia que en principio no tiene un límite tan cercano, pero que exige que la aplicación esté diseñada para funcionar en varias instancias a la vez sin depender de estado guardado localmente en una sola de ellas. Facebook documentó un ejemplo real de los desafíos que aparecen a esta escala: al construir su infraestructura de memcache para atender miles de millones de peticiones por segundo, tuvieron que resolver problemas que simplemente no existen en un sistema pequeño, como picos de tráfico coordinados sobre un mismo dato popular cuando su caché expira al mismo tiempo para todas las peticiones concurrentes [12]. Este tipo de problemas ilustra por qué la escalabilidad no es solamente "agregar más servidores", sino un conjunto de decisiones de diseño que deben tomarse con anticipación.

5.2 Patrones de caché en aplicaciones web

La caché es, junto con la escalabilidad horizontal, una de las herramientas más efectivas para que una aplicación web soporte más carga sin aumentar proporcionalmente el costo de infraestructura. El patrón más común es cache-aside: la aplicación consulta primero la caché, y solo si el dato no está ahí (cache miss) consulta la base de datos y guarda el resultado en caché para las siguientes peticiones [13]. Este patrón resulta especialmente adecuado cuando la base de datos sigue siendo la fuente de verdad y la caché es simplemente un acelerador desechable, ya que si la caché se pierde por completo, la aplicación sigue funcionando, solo que más lenta, consultando directamente la base de datos [13]. Redis es una de las tecnologías más usadas para implementar este patrón, gracias a que es un almacén de estructuras de datos en memoria diseñado para ofrecer tiempos de respuesta muy por debajo de los de una base de datos relacional en disco [14]. Un detalle importante en cualquier escenario con más de un servidor de aplicación es que la caché debe ser compartida

entre todas las instancias, no una copia local por servidor, porque de lo contrario cada instancia tendría una visión distinta y desactualizada de los mismos datos.

5.3 Documentación y evaluación de arquitecturas web

Una arquitectura bien diseñada pierde buena parte de su valor si nadie puede entender por qué se tomaron ciertas decisiones meses después de que el sistema ya está en producción. El modelo C4, propuesto por Simon Brown, documenta la arquitectura de un sistema en niveles progresivos de abstracción: desde un nivel de Contexto que muestra el sistema como una caja negra y sus actores externos, hasta niveles más detallados de Contenedores, Componentes y, opcionalmente, Código [15]. Esto permite comunicar la arquitectura al nivel de detalle adecuado según la audiencia, sin obligar a que cada diagrama muestre absolutamente todo el sistema a la vez. Complementando esto, un Architectural Decision Record (ADR) es un documento breve que registra una decisión arquitectónica junto con el contexto que la motivó, las alternativas consideradas, y las consecuencias esperadas; su valor no está en el resultado final —que en principio podría inferirse leyendo el código— sino en preservar el razonamiento detrás de esa decisión, algo que de otra forma se pierde con el tiempo y la rotación del equipo.

6 Conclusión

La programación del lado del servidor, más allá de qué tecnología específica se use, resuelve siempre los mismos problemas de fondo: representar una petición y una respuesta, recordar el estado de un usuario a pesar de que HTTP no lo haga por sí solo, comunicarse de forma segura y eficiente con una base de datos, y organizarse arquitectónicamente de una forma que permita crecer sin volverse imposible de mantener.

Los objetos integrados como Request, Response, Session y Application resuelven el primer problema, y aunque su nombre cambia entre PHP, Java y ASP.NET, el concepto detrás es prácticamente el mismo en las tres tecnologías. El acceso a datos, ya sea mediante SQL directo con sentencias preparadas, procedimientos almacenados, o un ORM como JPA, resuelve el segundo, con un compromiso constante entre la velocidad de desarrollo y el control fino sobre lo que realmente ocurre en la base de datos.

La arquitectura por capas resultó ser la opción más razonable para la mayoría de aplicaciones web de alcance moderado, reservando alternativas como los microservicios, SOA o las arquitecturas orientadas a eventos para escenarios donde el tamaño del equipo o del sistema realmente lo justifica. Finalmente, escalar una aplicación web exige diseño anticipado, no solo más servidores: la caché, bien aplicada con patrones como cache-aside, y una arquitectura documentada con herramientas como el modelo C4 y los ADR, son las piezas que permiten que un sistema crezca sin perder de vista por qué está construido como está.

7 Bibliografía

- [1] Eclipse Foundation, “Jakarta Servlet.” Accessed: Jul. 15, 2026. [Online]. Available: <https://jakarta.ee/learn/docs/jakartaee-tutorial/current/web/servlets/servlets.html>
- [2] The PHP Group, “Superglobals.” Accessed: Jul. 15, 2026. [Online]. Available: <https://www.php.net/manual/en/language.variables.superglobals.php>
- [3] Microsoft, “Session and app state in ASP.NET Core.” Accessed: Jul. 15, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state?view=aspnetcore-10.0>

- [4] Oracle, "Lesson: JDBC Basics." Accessed: Jul. 15, 2026. [Online]. Available: <https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html>
- [5] The PHP Group, "PDO." Accessed: Jul. 15, 2026. [Online]. Available: <https://www.php.net/manual/en/book.pdo.php>
- [6] PostgreSQL Global Development Group, "CREATE PROCEDURE." Accessed: Jul. 15, 2026. [Online]. Available: <https://www.postgresql.org/docs/current/sql-createprocedure.html>
- [7] Eclipse Foundation, "Jakarta Persistence 3.0 Specification." Accessed: Jul. 15, 2026. [Online]. Available: <https://jakarta.ee/specifications/persistence/3.0/jakarta-persistence-spec-3.0.html>
- [8] IEEE Computer Society, "Guide to the Software Engineering Body of Knowledge (SWEBOK), versión 4.0." Accessed: Jul. 15, 2026. [Online]. Available: <https://ieeecs-media.computer.org/media/education/swbok/swbok-v4.pdf>
- [9] D. Garlan and M. Shaw, "An Introduction to Software Architecture," 1994. Accessed: Jul. 15, 2026. [Online]. Available: http://sunnyday.mit.edu/16.355/intro_softarch.pdf
- [10] OASIS, "Reference Model for Service Oriented Architecture 1.0," 2006. Accessed: Jul. 15, 2026. [Online]. Available: <https://docs.oasis-open.org/soa-rm/v1.0/soa-rm.pdf>
- [11] M. Fowler, "Microservices: a definition of this new architectural term." Accessed: Jul. 15, 2026. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [12] R. Nishtala *et al.*, "Scaling Memcache at Facebook," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, 2013. Accessed: Jul. 15, 2026. [Online]. Available: https://www.usenix.org/system/files/conference/nsdi13/nsdi13-final170_update.pdf
- [13] Microsoft, "Cache-Aside pattern." Accessed: Jul. 15, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside>
- [14] Redis, "Understand Redis data types." Accessed: Jul. 15, 2026. [Online]. Available: <https://redis.io/docs/latest/develop/data-types/>
- [15] S. Brown, "The C4 model for visualising software architecture." Accessed: Jul. 15, 2026. [Online]. Available: <https://c4model.com>